מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את
מדבקת הנבחן

האוניברסיטה
הפתוחה

ד' בתשרי תשע"ט

מס' שאלון - 521

13

בספטמבר 2018

סמסטר 2018ב

מס' מועד 95

20905 / 4

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **13** עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (25 נקודות)

הרחיבו את שפת LET (שפת "ויהי") עם ביטוי חדש בשם letsum.

להלן הדקדוק המגדיר את התחביר החדש עבור ביטוי letsum :

***Expression* ::= letsum *identifier* = [{*Expression* }$^{*(,)}$]**
in [{*Expression* }$^{*(,)}$]

letsum-exp (id exps1 exps2)

כאשר,

- id הוא משתנה חדש המוכר רק במסגרת הביטוי.
- exps1 היא רשימה של אפס או יותר ביטויים המופרדים בפסיקים.
- exps2 היא רשימה של אפס או יותר ביטויים המופרדים בפסיקים.

שתי הרשימות, exps1 ו- exps2, אמורות להכיל את אותה כמות ביטויים. הביטוי letsum מחשב כל ביטוי מהרשימה exps2 עם id שערכו שווה לערך הביטוי מהמיקום התואם ברשימה exps1. כלומר, הביטוי הראשון ב-exps2 יחושב כאשר ערכו של המשתנה המצוין ע"י id שווה לערכו של הביטוי הראשון מהרשימה exps1, לאחר מכן יחושב ערכו של הביטוי השני מהרשימה exps2 כאשר ערכו של המשתנה המצוין ע"י id שווה לערכו של הביטוי השני מהרשימה exps1, וכן הלאה. ביטוי letsum יחזיר את סכום תוצאות חישוב הביטויים מהרשימה exps2.
במקרה של רשימות ריקות יוחזר אפס. במקרה של חוסר התאמה באורך הרשימות תודפס הודעת שגיאה. ניתן להניח שערכם של כל הביטויים ברשימות הוא מספרי.

להלן דוגמא להמחשת השימוש בביטוי החדש :

```
let x= 10
in
  let z=letsum x =[1,-(x,8),3] in [-(x,-1) , -(x,30),  -(x,-7)]
   in  -(z,x)
```

בדוגמה זו, בביטוי letsum הרשימה השנייה מכילה 3 ביטויים, הביטוי הראשון מחושב כאשר ערכו של x הוא 1. הביטוי השני מחושב כאשר ערכו של x הוא 2 (מכיוון שהביטוי השני ברשימה הראשונה הוא (x,8)– והוא מחושב על פי ה-x החיצוני שערכו 10).
ולבסוף הביטוי השלישי ברשימה השניה מחושב עם ערך של 3. ולכן בסה"כ ערכו של z יהיה שווה לסכום של הביטויים שחושבו ברשימה השניה. ולכן, z שווה ל- 16-.
הערך המוחזר של כל הביטוי המתואר בדוגמה הוא (z,x)– המחושב עם x שערכו 10. ולכן התשובה הכוללת שתוחזר היא (num-val -26)

א. (7 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של השפה כך שיתאים להגדרת הדקדוק החדש. **<u>את תשובותיכם כתבו אך ורק במחברת הבחינה.</u>**

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
    (comment ("%" (arbno (not #\newline))) skip)
    (identifier (letter (arbno (or letter digit "_" "-" "?"))) symbol)
    (number (digit (arbno digit)) number)
    (number ("-" digit (arbno digit)) number)
   ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
    (expression (number) const-exp)
    (expression ("-" "(" expression "," expression ")") diff-exp)
    (expression ("zero?" "(" expression ")") zero?-exp)
    (expression ("if" expression "then" expression "else" expression) if-exp)
    (expression (identifier) var-exp)
    (expression ("let" identifier "=" expression "in" expression) let-exp)
   ))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"ויהי"** כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את ביטוי letsum על פי הדקדוק החדש. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

```
;;value-of :  Exp x Env ----> ExpVal
 (define value-of
  (lambda (exp env)
   (cases expression exp
    (const-exp (num) (num-val num))
    (var-exp (var) (apply-env env var))
    (diff-exp (exp1 exp2)
       (let ((val1 (value-of exp1 env))
             (val2 (value-of exp2 env)))
         (let ((num1 (expval->num val1))
```

```
            (num2 (expval->num val2)))
        (num-val (- num1 num2)))))
(zero?-exp (exp1)
   (let ((val1 (value-of exp1 env)))
     (let ((num1 (expval->num val1)))
       (if (zero? num1) (bool-val #t)  (bool-val #f)))))
(if-exp (exp1 exp2 exp3)
   (let ((val1 (value-of exp1 env)))
      (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
(let-exp (var exp1 body)
   (let ((val1 (value-of exp1 env)))
      (value-of body (extend-env var val1 env))))
)))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

כזכור שפת PROC ("וישגור") מאפשרת הגדרת פרוצדורות בעלות פרמטר יחיד.

אם רוצים לבנות פרוצדורות עם יותר פרמטרים יש צורך להשתמש בטכניקה שנקראת Curried procedures (הייתה על כך שאלה בממנים). בטכניקה זו משתמשים לבניית פרוצדורות עם יותר מפרמטר אחד ע"י שימוש בכינון של פרוצדורות בעלות פרמטר יחיד. כלומר, בונים פרוצדורה שמקבלת פרמטר יחיד ומחזירה פרוצדורה אחרת שגם לה פרמטר יחיד (ושמכירה את הפרמטר של הפרוצדורה שיצרה אותה) וכך בעצם בונים פרוצדורה בעלת 2 פרמטרים ע"י פרוצדורות בעלות פרמטר יחיד.

להלן דוגמה, כיצד ניתן לבנות בשפת PROC (המקורית) פרוצדורה עם 2 פרמטרים באמצעות שימוש ב-Curring:

```
let p = proc (x)
        proc (y) –(x,y)
in ((p 10) 4)
```

בדוגמה זו, p היא פרוצדורה המקבלת פרמטר x ומחזירה פרוצדורה אחרת המקבלת פרמטר y (ושגם מכירה את x).

בהפעלת הפרוצדורה p, נשלח ארגומנט של 10 עבור הפרמטר x, בתמורה מוחזרת פרוצדורה אחרת שמצפה לפרמטר וגם אותה מפעילים עם ארגומנט שערכו 4. ולכן ערכה הסופי של תוכנית זו הוא 6.

בשאלה זו, ברצוננו להטמיע בהגדרות שפת PROC את היכולת להגדרת פרוצדורות בעלות 2 פרמטרים שנבנות ע"י Curring (ועושות שימוש רק בפרוצדורות בעלות פרמטר יחיד).

עליכם להרחיב את שפת PROC עם ביטוי חדש שתחבירו נתון על-ידי הדקדוק הבא:

Expression ::= **Curry (*identifier , identifier*)** ***Expression***

```
Curry-exp (id1 id2 body)
```

כאשר,

- id1 id2 הם פרמטרים עבור הפרוצדורה שתיבנה
- Body הוא ביטוי המתאר את גוף הפרוצדורה שתיבנה.

הסבר:

הפרוצדורה שתיבנה ע"י ביטוי Curry תיבנה בטכניקה שתוארה בתחילת השאלה.

שימו לב, לא יתקבל פתרון המגדיר בניית פרוצדורה עם 2 פרמטרים, וביטוי נוסף להפעלת פרוצדורה עם 2 פרמטרים. יש להשתמש רק ביכולות הקיימות בשפה של הגדרה והפעלה של פרוצדורות עם פרמטר יחיד.

להלן דוגמה להמחשת השימוש בביטוי החדש:

```
let p= Curry (x,y) –(x,y)
in
  ((p 10) 4)
```

דוגמה זו זהה לדוגמה שתוארה בתחילת השאלה, ובמקום שהמתכנת בשפה יממש בעצמו Curring, השפה עצמה תאפשר לו זאת.

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וישגור**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
      (define the-grammar
          '((program (expression) a-program)

            (expression (number) const-exp)
            (expression
              ("-" "(" expression "," expression ")")
      diff-exp)

            (expression
             ("zero?" "(" expression ")")
      zero?-exp)

            (expression
             ("if" expression "then" expression "else" expression)
      if-exp)

            (expression (identifier) var-exp)

            (expression
             ("let" identifier "=" expression "in" expression)
      let-exp)

            (expression
             ("proc" "(" identifier ")" expression)
      proc-exp)

             (expression
             ("(" expression expression ")")
      call-exp)
```

```
(expression
```

את התשובה יש לכתוב במחברת הבחינה

```
Curry-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"וישגור"**. ולאחריה פרוצדורת העזר apply-procedure כפי שמופיעה בספר הלימוד בעמוד 79. יש להשלים את החסר בפרוצדורה value-of. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;;value-of : Exp x Env→ExpVal
(define value-of
(lambda (exp env)
(cases expression exp
(const-exp (num) (num-val num))
(var-exp (var) (apply-env env var))
(diff-exp (exp1 exp2)
  (let ((val1 (value-of exp1 env))
(val2 (value-of exp2 env)))
(let ((num1 (expval->num val1))
   (num2 (expval->num val2)))
(num-val(- num1 num2)))))
(zero?-exp (exp1)
(let ((val1 (value-of exp1 env)))
(let ((num1 (expval->num val1)))
(if (zero? num1)(bool-val #t)(bool-val #f)))))
(if-exp (exp1 exp2 exp3)
 (let ((val1 (value-of exp1 env)))
 (if (expval->bool val1)
(value-of exp2 env)
(value-of exp3 env))))
       (let-exp (var exp1 body)
 (let ((val1 (value-of exp1 env)))
 (value-of body(extend-env var val1 env))))

     (proc-exp (var body)
          (proc-val (procedure var body env)))

(call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
(Curry-exp  (_________________________________)

                 את התשובה יש לכתוב במחברת הבחינה
                                                  ))))
```

```
;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
  (define apply-procedure
    (lambda (proc1 val)
      (cases proc proc1
        (procedure (var body saved-env)
          (value-of body (extend-env var val saved-env))))))
```

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "**וירמוז**"(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם ChangeBody.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression ::= ChangeBody (*identifier* , *Expression*)

ChangeBody-exp (prc newbody)

לביטוי **ChangeBody-exp** המרכיבים הבאים:

- **prc** – שם של פרוצדורה שכבר הוגדרה קודם לכן.
- **Newbody** – ביטוי שיחליף את גוף הפרוצדורה שהוגדרה.

ביטוי ChangeBody אמור להחליף לפרוצדורה שכבר הוגדרה, את גוף הפרוצדורה בגוף חדש, כל יתר פרטי הפרוצדורה יוותרו ללא שינוי. כך שמעתה אם הפרוצדורה תופעל היא תבצע את גוף הפרוצדורה החדש. ביטוי זה אינו מחזיר תשובה, והוא דומה לביטוי השמה המחזיר ערך פיקטיבי.

להלן דוגמא:

```
let dec = proc (x) -(x,1)
in begin
     ChangeBody(dec , -(x,2));
     (dec 8)
   end
```

בדוגמה זו, מוגדרת פרוצדורה בשם dec המקבלת פרמטר x ומחזירה x-1.

לאחר מכן, משנים את גוף הפרוצדורה בלבד לביטוי חדש x-2.

ולבסוף מופעלת הפרוצדורה על ארגומנט שערכו 8. ולכן ערכה הסופי של התוכנית הוא

(num-val 6)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וירמוז**". עליכם להשלים את החסר בריבוע המסומן. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
   ("if" expression "then" expression "else" expression)
     if-exp)
  (expression (identifier) var-exp)
  (expression ("let" identifier "=" expression "in" expression)
     let-exp)
  (expression ("proc" "(" identifier ")" expression)
     proc-exp)
  (expression ("(" expression expression ")") call-exp)
  (expression
        ("letrec"
          (arbno identifier "(" identifier ")" "=" expression)
           "in" expression)
letrec-exp)
(expression ("set" identifier "=" expression)
    assign-exp)
(expression (
```

את התשובה כיתבו במחברת הבחינה

```
)
  ChangeBody-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו- *apply-procedure* של שפת **"וירמוז"**.

הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

<u>את התשובה כיתבו במחברת הבחינה</u>.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
  (define value-of
    (lambda (exp env)
      (cases expression exp
        (const-exp (num) (num-val num))

       (var-exp (var) (deref (apply-env env var)))

        (diff-exp (exp1 exp2)
          (let ((val1 (value-of exp1 env))
                (val2 (value-of exp2 env)))
            (let ((num1 (expval->num val1))
                  (num2 (expval->num val2)))
              (num-val
                (- num1 num2)))))

        (zero?-exp (exp1)
          (let ((val1 (value-of exp1 env)))
            (let ((num1 (expval->num val1)))
              (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

        (if-exp (exp1 exp2 exp3)
          (let ((val1 (value-of exp1 env)))
            (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

        (let-exp (var exp1 body)
          (let ((v1 (value-of exp1 env)))
            (value-of body
              (extend-env var (newref v1) env))))

        (proc-exp (var body)
          (proc-val (procedure var body env)))

        (call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
```

```
                (arg (value-of rand env)))
            (apply-procedure proc arg)))


      (letrec-exp (p-names b-vars p-bodies letrec-body)
          (value-of letrec-body
            (extend-env-rec* p-names b-vars p-bodies env)))

        (assign-exp (var exp1)
          (begin
            (setref!
              (apply-env env var)
              (value-of exp1 env))
            (num-val 27))))))


(define apply-procedure
      (lambda (proc1 val)
         (cases proc proc1
            (procedure (var body saved-env)
                (value-of body
                   (extend-env var (newref val) saved-env))))))
```

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p = (proc (w:?) -(w,7)  9)
in
if zero? (p) then p else (proc (y:?) -(y,7)  20)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !